

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-af9e07c30569178dd.jsonl`  
- SHA-256: `29ff9876cda772018fd10f58f4fb39ebb0e9fdbba189ec9346ff58e6d7955398`  
- Source modified: `2026-07-06T06:11:49+00:00`  
- Imported at: `2026-07-08T16:00:13+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:09:54.706Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension adr_conformance):

title: policies single-default UNIQUE partial index is specified by ADR 0011 but never created
severity(claimed): medium
file:line: src/tg_video_filter/db.py:124
summary: The ADR (and models.py) promise 'is_default ... one per account (UNIQUE partial idx)', but _SCHEMA creates only UNIQUE(account_id, name); nothing creates a partial unique index on

is_default, so multiple default policies per account are admitted and get_default_policy becomes non-deterministic.

failure_scenario: For account 111, save_filter_config creates the name='default', is_default=1 policy. A caller then does upsert_policy(Policy(account_id=111, name='reggae-route', is_default=True)) (a per-route override, the exact use case ADR 0011 Option 2A cites). Both rows now have is_default=1. get_default_policy runs 'SELECT * FROM policies WHERE account_id=? AND is_default=1' with no ORDER BY / LIMIT (db.py:1068-1071) and returns whichever row SQLite yields first ? the filter engine can silently load the wrong policy. The DB does not reject the second default.

evidence: Repro (venv): PRAGMA index_list('policies') ? [('sqlite_autoindex_policies_1', 1)] (only the UNIQUE(account_id,name) autoindex). Inserting two is_default=1 rows for account 111 succeeds; 'default policies for account 111: 2 (ADR says must be <=1)'. ADR 0011:141 '-- one per account (UNIQUE partial idx)'; models.py:205 'Exactly one per account (UNIQUE partial index)'. db.py:1034-1035 admits it: 'enforced by a partial unique index landed by future work, not in the schema'. Grep for 'idx_policies'/'CREATE UNIQUE INDEX ... policies' ? no matches.

recommendation: Add 'CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_one_default ON policies(account_id) WHERE is_default = 1' to _SCHEMA (and, if legacy DBs matter, to migration 003), matching the idx_videos_user_document pattern already used at db.py:338. Alternatively, if enforcement is genuinely deferred, remove the 'UNIQUE partial idx' claim from the ADR/models docstring so the spec doesn't over-promise.

2. assistant (2026-07-06T06:09:58.140Z)

I'll verify this finding by reading the actual code at the cited lines.

3. user (2026-07-06T06:09:59.650Z)

```
100
101     CREATE TABLE IF NOT EXISTS content_items (
102         id                INTEGER PRIMARY KEY AUTOINCREMENT,
103         account_id        INTEGER NOT NULL REFERENCES accounts(id),
104         source_id          INTEGER NOT NULL REFERENCES sources(id),
105         provider           TEXT NOT NULL DEFAULT 'telegram',
106         provider_message_id TEXT NOT NULL,
107         content_type       TEXT NOT NULL DEFAULT 'video',
108         text               TEXT,
109         link_url           TEXT,
110         media_ref          TEXT,
111         document_id        TEXT,
112         duration_s         INTEGER,
113         width              INTEGER,
114         height             INTEGER,
115         size_bytes         INTEGER,
116         mime_type          TEXT,
117         dedup_key          TEXT NOT NULL,
118         matched            INTEGER NOT NULL DEFAULT 0,
119         seen_at            TEXT NOT NULL,
120         UNIQUE (account_id, source_id, provider_message_id),
121         UNIQUE (account_id, dedup_key)
122     );
123
124     CREATE TABLE IF NOT EXISTS policies (
125         id                INTEGER PRIMARY KEY AUTOINCREMENT,
126         account_id        INTEGER NOT NULL REFERENCES accounts(id),
127         name               TEXT NOT NULL,
128         config_json        TEXT NOT NULL,
129         is_default         INTEGER NOT NULL DEFAULT 0,
130         created_at         TEXT NOT NULL,
131         updated_at         TEXT NOT NULL,
132         UNIQUE (account_id, name)
133     );
134
135     CREATE TABLE IF NOT EXISTS destinations (
136         id                INTEGER PRIMARY KEY AUTOINCREMENT,
137         account_id        INTEGER NOT NULL REFERENCES accounts(id),
```

```

138     provider    TEXT NOT NULL DEFAULT 'telegram',
139     target      TEXT NOT NULL,
140     kind        TEXT NOT NULL DEFAULT 'saved',
141     title       TEXT,
142     is_output    INTEGER NOT NULL DEFAULT 0,
143     created_at   TEXT NOT NULL,
144     UNIQUE (account_id, provider, target)
145 );
146
147 CREATE TABLE IF NOT EXISTS routing_rules (
148     id           INTEGER PRIMARY KEY AUTOINCREMENT,
149     account_id   INTEGER NOT NULL REFERENCES accounts(id),
150     source_id    INTEGER NOT NULL REFERENCES sources(id),
151     policy_id    INTEGER REFERENCES policies(id),
152     destination_id INTEGER NOT NULL REFERENCES destinations(id),
153     enabled      INTEGER NOT NULL DEFAULT 1,
154     priority     INTEGER NOT NULL DEFAULT 0,
155     created_at   TEXT NOT NULL,
156     UNIQUE (account_id, source_id, destination_id)
157 );
158
159 CREATE TABLE IF NOT EXISTS delivery_attempts (

```

4. user (2026-07-06T06:10:00.275Z)

```

1020         (account_id, dedup_key),
1021     ) as cur:
1022         row = await cur.fetchone()
1023         return _row_to_content_item(row) if row else None
1024
1025
1026 # --- Policy repos (ADR 0011, Option 2A) -----
1027
1028
1029 async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1030     """Insert or update a policy row by ``(account_id, name)``.
1031
1032     Exactly one policy per account should carry ``is_default=True``; callers
1033     are responsible for flipping the previous default before flipping a new
1034     one (enforced by a partial unique index landed by future work, not in the
1035     schema ? SQLite ``CHECK`` can't express "at most one per group").
1036     """
1037     await conn.execute(
1038         """INSERT INTO policies (account_id, name, config_json, is_default,
1039                                created_at, updated_at)
1040         VALUES (?, ?, ?, ?, ?, ?)
1041         ON CONFLICT(account_id, name) DO UPDATE SET
1042             config_json=excluded.config_json,
1043             is_default=excluded.is_default,
1044             updated_at=excluded.updated_at""",
1045         (
1046             policy.account_id,
1047             policy.name,
1048             policy.config_json,
1049             int(policy.is_default),
1050             policy.created_at.isoformat(),
1051             policy.updated_at.isoformat(),
1052         ),
1053     )
1054     await conn.commit()
1055     conn.row_factory = aiosqlite.Row
1056     async with conn.execute(
1057         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1058         (policy.account_id, policy.name),
1059     ) as cur:

```

```

1060         row = await cur.fetchone()
1061         assert row is not None
1062         return _row_to_policy(row)
1063
1064
1065     async def get_default_policy(conn: aiomysqlite.Connection, account_id: int) -> Policy |
None:
1066         """Return the account's default policy (`is_default=1`)."""
1067         conn.row_factory = aiomysqlite.Row
1068         async with conn.execute(
1069             "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1070             (account_id,),
1071         ) as cur:
1072             row = await cur.fetchone()
1073             return _row_to_policy(row) if row else None
1074
1075
1076     # --- legacy Channel shim (backwards-compatible view over sources) -----
1077
1078
1079     async def upsert_channel(conn: aiomysqlite.Connection, channel: Channel) -> Channel:
1080         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1081
1082         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1083         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1084         New code should call :func:`upsert_source` directly.
1085         """
1086         src = await upsert_source(
1087             conn,
1088             Source(
1089                 account_id=channel.user_id,
1090                 provider="telegram",
1091                 provider_source_id=str(channel.channel_id),
1092                 title=channel.title,
1093                 kind="channel",
1094                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1095                 seen_first_at=channel.seen_first_at,
1096             ),
1097         )
1098         return _source_to_channel(src)
1099

```

5. assistant (2026-07-06T06:10:05.120Z)

The finding is largely a documentation/ADR conformance issue. Let me verify the key claims: (1) no partial unique index exists, (2) the ADR and models.py promise one, and (3) the failure scenario is reachable.

6. user (2026-07-06T06:10:07.224Z)

```

339:         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
485:         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "

```

7. user (2026-07-06T06:10:08.706Z)

...

```

policies (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id    INTEGER NOT NULL REFERENCES accounts(id),
    name          TEXT NOT NULL,                    -- e.g. "default", "reggae-route"
    config_json   TEXT NOT NULL,                    -- the FilterConfig blob (Phase 1)
    is_default    INTEGER NOT NULL DEFAULT 0,       -- one per account (UNIQUE partial idx)
    created_at    TEXT NOT NULL,
    updated_at    TEXT NOT NULL,

```

```

        UNIQUE (account_id, name)
    );
    ...

```

****Phase 1**** (this track): `config\_json` holds exactly today's `FilterConfig` JSON ? the rename is purely structural. ****Phase 2**** (Track F, deferred): `config\_json` grows to include topic preferences and security rules, but

8. user (2026-07-06T06:10:08.779Z)

```

195
196     model_config = ConfigDict(frozen=True)
197
198     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199     account_id: int = Field(description="FK ? accounts.id")
200     name: str = Field(description="Policy name; 'default' for the account default
policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     class FilterConfig(BaseModel):
212         """Per-user filter configuration.
213
214         v1 rules: duration bounds, minimum height (resolution), and max file size.

```

9. assistant (2026-07-06T06:10:13.589Z)

All facts verified. Now let me check the ADR wording precisely ? is the "UNIQUE partial idx" a hard promise or an annotation of intent ? and confirm the failure scenario is reachable via a quick repro.

10. user (2026-07-06T06:10:15.176Z)

```

3:- **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
60:first and each branch is **guarded so partial metadata can never collapse
75:partial-metadata collision the first draft of this ADR allowed. The pure
79:partial/null metadata.
81:A **partial unique index** on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
110:deep-inspect stage (deferred to Track F per #31).
129:#### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
141:     is_default     INTEGER NOT NULL DEFAULT 0,          -- one per account (UNIQUE partial idx)
149:JSON ? the rename is purely structural. **Phase 2** (Track F, deferred):
154:`name='default'`, `is_default=1`, `config_json`=<the existing blob>`. Drop
164: become thin shims over `policies WHERE account_id=? AND is_default=1`,
211:flag (ADR 0009, deferred to Track E) is included here so Track E doesn't
255:### `policies` (generalizes `filter_config`) ? Option 2A
333:     `COALESCE(...,0)` collapsing partial metadata to a degenerate key):
338:     - Else (Tier-0 fallback; partial or missing media metadata):
347:     `name='default'`, `is_default=1`, `config_json` copied verbatim.
367:- `Policy` (Option 2A) gives a single home for filter + future enrichment

```

11. assistant (2026-07-06T06:10:27.736Z)

Interesting ? line 81 shows the ADR *does* explicitly create a partial unique index for `dedup\_key` (which the schema implements via UNIQUE(account_id, dedup_key) at db.py:121 ? though note that's a full unique index, not partial). Let me check the `policies` section around line 255 and whether the schema shown there includes an index, and check line 347 for the

`save\_filter\_config` reference.

12. user (2026-07-06T06:10:29.267Z)

`policies` (generalizes `filter\_config`) ? Option 2A

As specified in Question 2 above.

`destinations` (new ? Track C will populate)

Defined here so the schema is complete, but **Track C (#28) implements the delivery layer**. Phase-1 presence is the table only (no delivery logic).

...

```
destinations (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    account_id    INTEGER NOT NULL REFERENCES accounts(id),  
    provider      TEXT NOT NULL DEFAULT 'telegram',  
    target        TEXT NOT NULL,          -- 'me', '@username', chat id (ADR 0005)  
    kind          TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'  
    title         TEXT,  
    is_output     INTEGER NOT NULL DEFAULT 0,      -- ADR 0009: distinguishes output channels  
    created_at    TEXT NOT NULL,  
    UNIQUE (account_id, provider, target)  
);  
...
```

`routing\_rules` (new ? Track C will populate)

Per ADR 0009 (canonical name locked in issue #25).

...

```
routing_rules (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    account_id    INTEGER NOT NULL REFERENCES accounts(id),  
    source_id     INTEGER NOT NULL REFERENCES sources(id),  
    policy_id     INTEGER,                -- nullable ? use account's default policy  
    destination_id INTEGER NOT NULL REFERENCES destinations(id),  
    enabled       INTEGER NOT NULL DEFAULT 1,  
    priority      INTEGER NOT NULL DEFAULT 0,  
);
```

13. user (2026-07-06T06:10:30.293Z)

```
129:     is_default INTEGER NOT NULL DEFAULT 0,  
571:     # (Option 2A). is_default=1 so the legacy load_filter_config shim reads  
575:     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,  
721:     is_default=bool(row["is_default"]),  
1032:     Exactly one policy per account should carry ``is_default=True``; callers  
1038:     """INSERT INTO policies (account_id, name, config_json, is_default,  
1043:     is_default=excluded.is_default,  
1049:     int(policy.is_default),  
1066:     """Return the account's default policy (``is_default=1``)."""  
1069:     "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",  
1281:     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``  
1290:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)  
-> None:  
1300:         is_default=True,  
1376:     "save_filter_config",
```

14. assistant (2026-07-06T06:10:34.440Z)

Now let me confirm the failure scenario with a repro: seed a default via save_filter_config, then upsert a second is_default policy, and check that get_default_policy is non-deterministic / the DB admits two.


```

1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aiosqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid
1317            FROM content_items ci
1318            JOIN sources s ON s.id = ci.source_id
1319            WHERE ci.account_id = ?

```

21. assistant (2026-07-06T06:11:23.184Z)

This confirms the analysis. `save_filter_config` always keys on `name="default"` (via `ON CONFLICT(account_id,name))`, so the deprecated shim alone can never create a second default ? it's idempotent on the "default" name. The two-default situation only arises when someone calls `upsert_policy` directly with a *different* name and `is_default=True`. That is exactly the "reggae-route" per-route override the ADR's Option 2A cites as a use case.

Now the crucial adversarial point on **severity** and whether wrong behaviour ships:

- `upsert_policy` and `get_default_policy` are **new** in this PR and callable. Nothing in the diff prevents two defaults.
- **However**, is there any current caller in the codebase that passes a non-"default" name with `is_default=True`? The only production caller of the default-policy path is `save_filter_config` (name="default"). Track C (routing) is explicitly deferred (ADR: "Track C will populate"). So today, no shipped code path actually creates two defaults ? the trigger requires a future caller.

Let me confirm there are no other callers of `upsert_policy` in the changed/existing code.

22. user (2026-07-06T06:11:24.913Z)

```

src/tg_video_filter/db.py:1284: policy = await get_default_policy(conn, user_id)
src/tg_video_filter/db.py:1292: existing = await get_default_policy(conn, user_id)
src/tg_video_filter/db.py:1293: await upsert_policy(
src/tg_video_filter/db.py:1361:     "upsert_policy",
src/tg_video_filter/db.py:1362:     "get_default_policy",
Binary file src/tg_video_filter/__pycache__/db.cpython-313.pyc matches
tests/test_db.py:1084:     """Upsert a default policy and read it back via get_default_policy."""
tests/test_db.py:1086:     policy = await db.upsert_policy(
tests/test_db.py:1096:     fetched = await db.get_default_policy(conn, USER_ID)
tests/test_db.py:1111:     policy = await db.get_default_policy(conn, USER_ID)
Binary file tests/__pycache__/test_db.cpython-313-pytest-8.4.2.pyc matches

```

23. assistant (2026-07-06T06:11:40.281Z)

Confirmed. The only production caller passing `is_default=True` is `save_filter_config` (name="default", idempotent). `upsert_policy` is exported in `__all__` (public API) and used by tests, but no shipped code path today creates a second default. The trigger requires future Track C routing code to call `upsert_policy(name="reggae-route", is_default=True)`.

Verdict assessment:

The finding's factual claims are all CONFIRMED:

- The partial unique index is not created (repro: only the autoindex exists).
- The ADR (line 141) and models.py (line 205) describe it as a property of the schema.
- The DB admits two defaults; `get_default_policy` has no ORDER BY/LIMIT and is non-deterministic.

But the *severity* is the contested part. The db.py docstring at 1034-1035 **explicitly** and truthfully documents that enforcement is "landed by future work, not in the schema." So within db.py the deferral is honest. The over-promise is narrower: models.py:205 states "Exactly one per account (UNIQUE partial index)" as a fact without the deferral caveat, and the ADR schema comment implies the idx exists. No shipped code path today produces two defaults (save_filter_config is idempotent on name="default"; Track C is deferred). So the "wrong

behaviour" (filter engine loading wrong policy) is latent, not currently reachable through any real caller.

This is a real doc/spec-vs-implementation conformance gap that ships in this diff (the schema, models docstring, and ADR are all in scope), but the concrete wrong-behaviour trigger cannot occur with current callers ? it requires future code. Severity **medium** as claimed is slightly high given no live trigger; **low** is more honest (latent/hygiene: spec over-promises, no current data corruption). The defect genuinely ships in the diff (an unenforced invariant that the public ``upsert_policy`` API will silently violate), so **CONFIRMED** rather than **REFUTED**.

24. user (2026-07-06T06:11:49.107Z)

Structured output provided successfully